Bit manipulation

23M. Bit Manipulation (5 problems)

Pattern recognition: XOR for pairing, single bits, powers of two, counting set bits.

23.163 Single Number (XOR)

`a ^ a = 0`, `a ^ 0 = a`. XOR all elements — pairs cancel, single remains.

JAVASCRIPT Copy

```javascript
function singleNumber(nums) {
  return nums.reduce((acc, num) => acc ^ num, 0);
}
```

23.164 Number of 1 Bits (Hamming Weight)

JAVASCRIPT Copy

```javascript
function hammingWeight(n) {
  let count = 0;
  while (n) { count += n & 1; n >>>= 1; }
  return count;
}
// Or: n & (n-1) removes lowest set bit — count iterations until n = 0
```

Approach: Use the n & (n - 1) trick, which clears the lowest set bit; a power of two has exactly one set bit, so the result is zero.

JAVASCRIPT Copy

```
function isPowerOfTwo(n) {
  return n > 0 && (n & (n - 1)) === 0;
}
```

A power of two has exactly one set bit: `n > 0 && (n & (n - 1)) === 0`.

23.166 Counting Bits (for 0 to n)

Approach: Dynamic programming with dp[i] = dp[i >> 1] + (i & 1), reusing the bit count of i with its lowest bit dropped.

JAVASCRIPT Copy

```
function countBits(n) {
  const dp = new Array(n + 1).fill(0);
  for (let i = 1; i ≤ n; i++) {
    dp[i] = dp[i >> 1] + (i & 1);
  }
  return dp;
}
```

`dp[i] = dp[i >> 1] + (i & 1)`. Number of 1-bits in i = bits in i/2 + whether last bit is set.

23.167 Missing Number (0 to n, one missing)

Approach: XOR all indices, all values, and n together; matching pairs cancel out and the leftover is the missing number.

JAVASCRIPT Copy

```
function missingNumber(nums) {
  let missing = nums.length;
  for (let i = 0; i < nums.length; i++) {
    missing ^= i ^ nums[i];
  }
  return missing;
}
```

XOR 0..n with all array elements — pairs cancel, missing remains. Or: `n*(n+1)/2 - arraySum`.

QUICK CHECK — no marks, just for you.

1. What does this snippet print?

JAVASCRIPT Copy

```
function singleNumber(nums) {
  return nums.reduce((acc, num) => acc ^ num, 0);
}
console.log(singleNumber([4, 1, 2, 1, 2]));
```

- ◯ 1
- ◯ 0
- ◯ 10
- ◯ 4

Check answer

JAVASCRIPT Copy

```
function hammingWeight(n) {
  let count = 0;
  while (n) { count += n & 1; n >>>= 1; }
  return count;
}
console.log(hammingWeight(11));
console.log(16 > 0 && (16 & (16 - 1)) === 0);
console.log(12 > 0 && (12 & (12 - 1)) === 0);
```

- 3
  true
  false
- 4
  true
  false
- 3
  false
  true
- 2
  true
  true

Check answer

3. For the Missing Number problem (an array holding all but one of the values 0 through n), why does XOR-ing 0..n together with every array element leave the missing value?

- XOR behaves like addition here, so the result is the arithmetic difference between the range total and the array total
- XOR returns the largest value in the range that does not appear in the array

Every value present in both the range and the array is XOR-ed twice and cancels to 0, so only the missing value remains

- ○ XOR only cancels duplicates that are adjacent, so the array must be sorted before the trick works

Check answer

‹ Previous
Strings

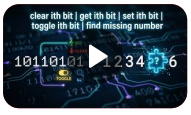
Next ›
Math

LEARN MORE

ARTICLES & DOCS

Bitwise algorithms (GeeksforGeeks)
geeksforgeeks.org

VIDEOS



Bit manipulation: 5 must-know tricks
YouTube



Master bit manipulation
YouTube



Bit manipulation (HackerRank)
YouTube

Hand-picked references for this lesson.